

16주차 예습과제

8장. 성능 최적화 (PyTorch)

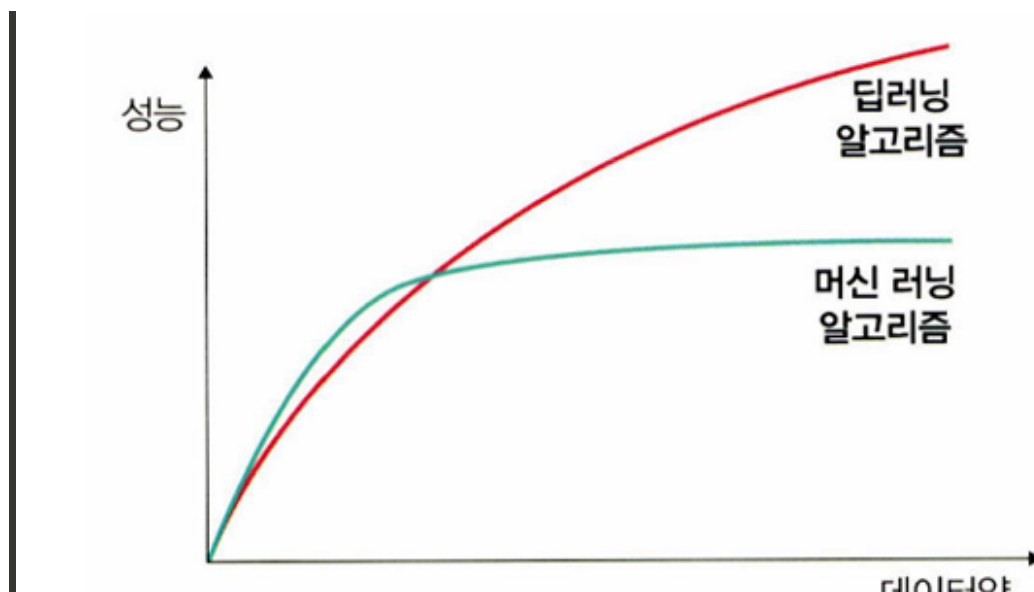
8.1 성능 최적화

딥러닝에는 성능을 최적화할 수 있는 다양한 방법이 있습니다. 데이터, 알고리즘, 알고리즘 튜닝, 앙상블 등 여러 관점에서 하나씩 살펴봅니다.

8.1.1 데이터를 사용한 성능 최적화

데이터를 사용한 성능 최적화의 핵심은 **많은 데이터를 수집하는 것**입니다. 하지만 데이터 수집이 여의치 않다면 임의로 데이터를 생성하는 방법도 고려할 수 있습니다.

- **최대한 많은 데이터 수집하기:** 일반적으로 딥러닝이나 머신러닝 알고리즘은 데이터양이 많을수록 성능이 좋아집니다. 따라서 가능한 많은 데이터(빅데이터)를 수집해야 합니다.



- **데이터 생성하기:** 많은 데이터를 수집할 수 없다면 데이터를 만들어 사용할 수 있습니다. (이미지 조작을 통한 데이터 증강 — 5장 참고)
- **데이터 범위(scale) 조정하기:** 활성화 함수에 따라 데이터셋 범위를 맞춰주는 것도 중요합니다.
 - 시그모이드 사용 시 → 데이터셋 범위 **0~1**
 - 하이퍼볼릭 탄젠트 사용 시 → 데이터셋 범위 **-1~1**

추가로 정규화, 규제화, 표준화도 성능 향상에 도움이 됩니다.

8.1.2 알고리즘을 이용한 성능 최적화

머신러닝과 딥러닝을 위한 알고리즘은 매우 다양합니다. 우리가 처음 선택한 알고리즘이 최적은 아닐 수 있으므로, 유사한 용도의 알고리즘들을 비교 훈련해서 최적의 성능을 보이는 것을 선택해야 합니다.

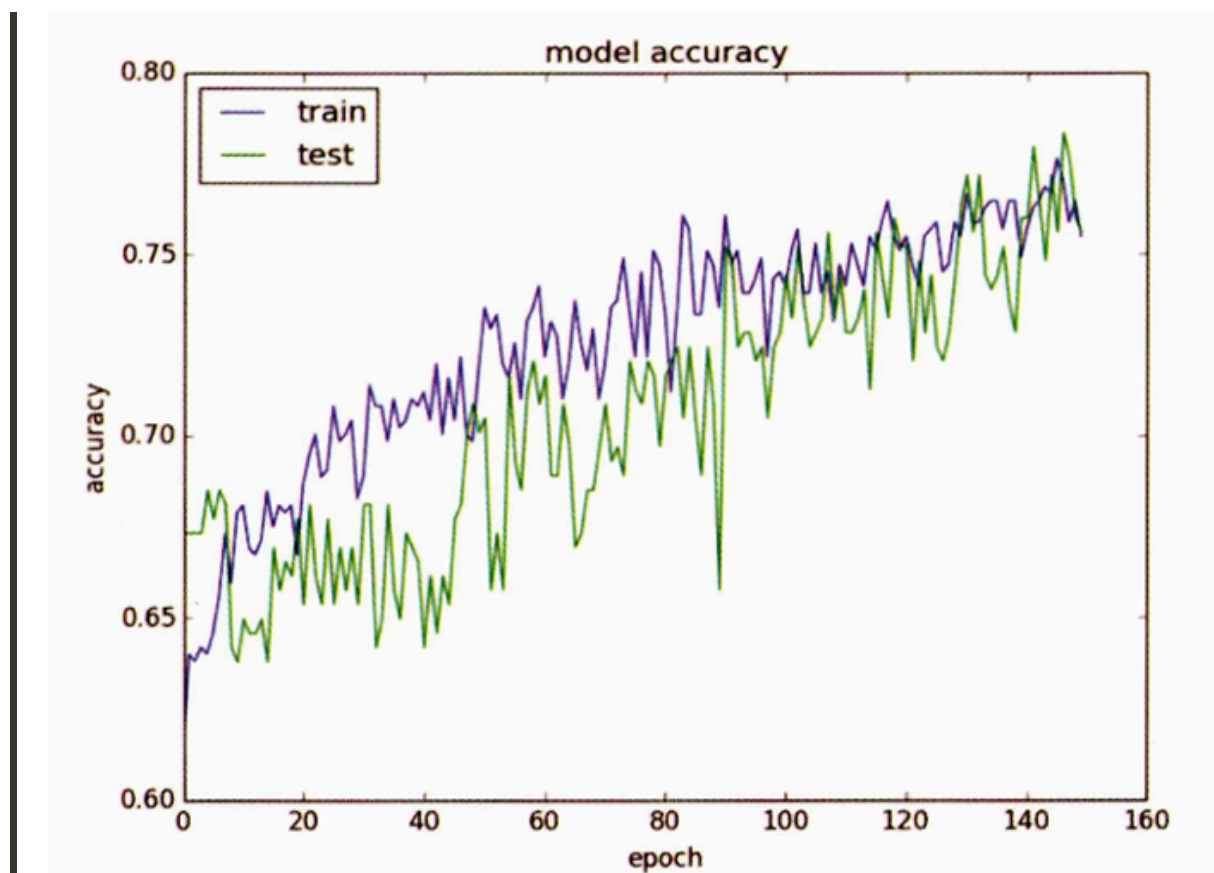
- **데이터 분류:** SVM, K-최근접 이웃 등을 비교
- **시계열 데이터:** RNN, LSTM, GRU 등을 비교 (7장 참고)

8.1.3 알고리즘 튜닝을 위한 성능 최적화

성능 최적화 중 **가장 많은 시간이 소요되는 부분**입니다. 다양한 하이퍼파라미터를 변경하며 훈련을 반복하고 최적의 성능을 도출해야 합니다.

진단 (Diagnostics)

성능 향상이 멈췄다면 원인을 분석해야 합니다. 훈련/검증 데이터에 대한 모델 평가 결과를 바탕으로 과적합 여부나 다른 원인을 진단할 수 있습니다.



진단 시 고려할 상황:

- **훈련 성능 >> 검증 성능:** 과적합(over-fitting) 의심 → 규제화로 해결 가능 (8.3.1절 참고)
- **훈련/검증 둘 다 성능이 안 좋음:** 과소적합(under-fitting) 의심 → 네트워크 구조 변경, 에포크 수 조정
- **훈련 성능이 검증을 넘어서는 변곡점 존재:** 조기 종료 고려

하이퍼파라미터 항목들

항목	설명
가중치	초깃값은 작은 난수 사용. 애매하면 오토인코더 등 비지도 학습으로 사전 훈련 후 지도 학습 진행
학습률	네트워크 계층이 많으면 학습률은 높게, 계층이 적으면 낮게 설정
활성화 함수	변경 시 손실 함수도 함께 변경해야 하는 경우가 많아 신중해야 함. 일반적으로 시그모이드/하이퍼볼릭 탄젠트 사용 시 출력층에서는 소프트맥스나 시그모이드 사용
배치와 에포크	큰 에포크 + 작은 배치가 최근 트렌드. 다양한 배치 크기로 테스트 권장
옵티마이저 및 손실 함수	일반적으로 확률적 경사 하강법(SGD) 많이 사용. Adam, RMSProp도 좋은 성능을 보임
네트워크 구성(토폴로지)	은닉층 뉴런 수를 늘리거나(넓은 네트워크), 계층을 늘리고 뉴런 수는 줄이는(깊은 네트워크) 방식으로 테스트

8.1.4 앙상블을 이용한 성능 최적화

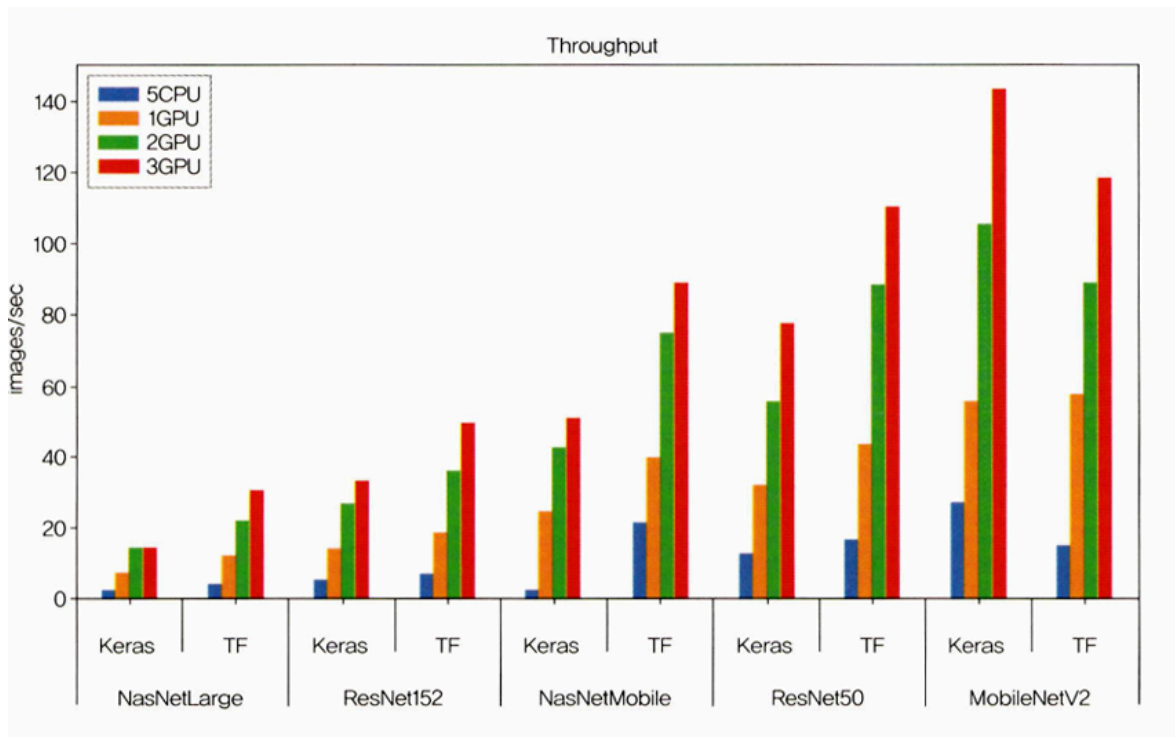
앙상블은 모델을 두 개 이상 섞어서 사용하는 것으로, 성능 향상에 도움이 됩니다.

알고리즘 튜닝은 하이퍼파라미터 조합의 수가 많아 수십~수백 번의 모델 훈련이 필요할 수 있습니다. 성능 향상은 단시간에 해결되지 않고 수많은 시행착오를 거쳐야 합니다.

8.2 하드웨어를 이용한 성능 최적화

8.2.1 CPU와 GPU 사용의 차이

딥러닝에서 GPU를 사용하면 분석 시간이 크게 단축됩니다.

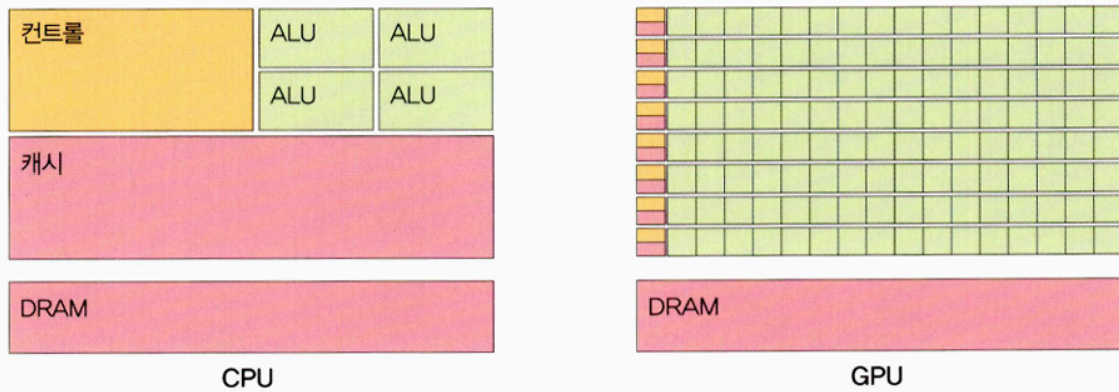


그래프를 보면 **CPU 5개를 동시에 돌려도 GPU 1개보다 성능이 낮습니다.** 이는 CPU와 GPU의 개발 목적과 내부 구조가 다르기 때문입니다.

CPU vs GPU 구조 비교

구분	CPU	GPU
구성	ALU(연산) + 컨트롤(명령어 해석/실행) + 캐시	많은 수의 ALU 중심
처리 방식	직렬 처리 (한 번에 하나의 명령어)	병렬 처리 (여러 명령어 동시 처리)
ALU 개수	적음	코어당 수백~수천 개
적합한 작업	순차적 연산 (예: 행렬 A·B·C 순서대로 처리)	병렬 연산 (예: 역전파의 복잡한 미적분)

▼ 그림 8-4 CPU와 GPU의 구조 비교



핵심 요약: 개별 코어 속도는 CPU가 GPU보다 빠르지만, 딥러닝처럼 수백~수천만 건의 데이터를 벡터로 변환해 병렬 연산해야 하는 경우는 GPU가 압도적으로 유리합니다. 딥러닝에서 GPU 사용은 **선택이 아닌 필수**입니다.

8.2.2 GPU를 이용한 성능 최적화

GPU 모델별 코어 정보 (표 8-1)

구분	GTX 1080	GTX 1070	GTX 1060(3GB)	GTX 1050
Core	2560	1920	11512	640
Clock(MHz)	1607	1506	1506	1354
Memory	11GB	8GB	6GB	4GB
Memory Speed	10Gbps	8Gbps	8Gbps	7Gbps

CUDA 툴킷 설치

CUDA 툴킷 - 드라이버 버전 호환표 (표 8-2, 일부)

CUDA 툴킷	리눅스 x86_64	윈도 x86_64
CUDA 11.0.189 RC	≥450.36.06	≥451.22
CUDA 10.2.89	≥440.33	≥441.22
CUDA 10.1	≥418.39	≥418.96

8.3 하이퍼파라미터를 이용한 성능 최적화

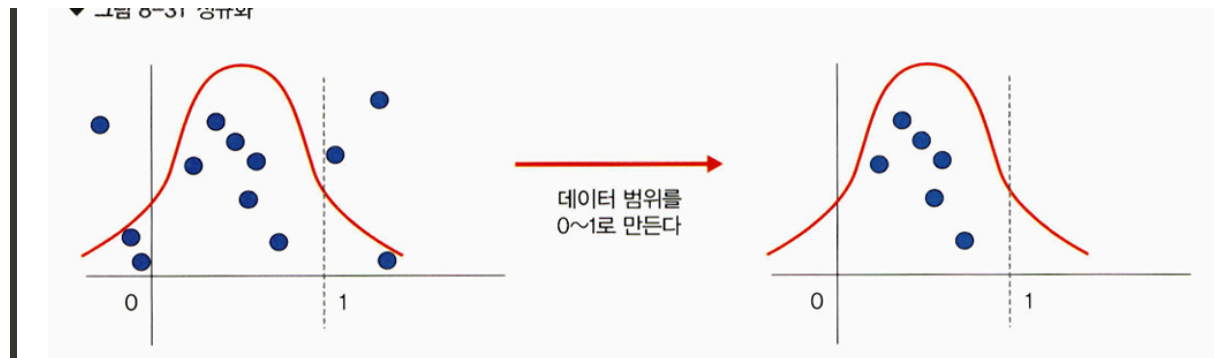
추가적인 최적화 방법으로 배치 정규화, 드롭아웃, 조기 종료를 다룹니다.

8.3.1 배치 정규화를 이용한 성능 최적화

관련 용어 정리

정규화(Normalization)

데이터 범위를 사용자가 원하는 범위로 제한하는 것 (예: 픽셀값 0~255를 255로 나눠 0~1.0으로 변환). 특성 스케일링(feature scaling)이라고도 합니다.

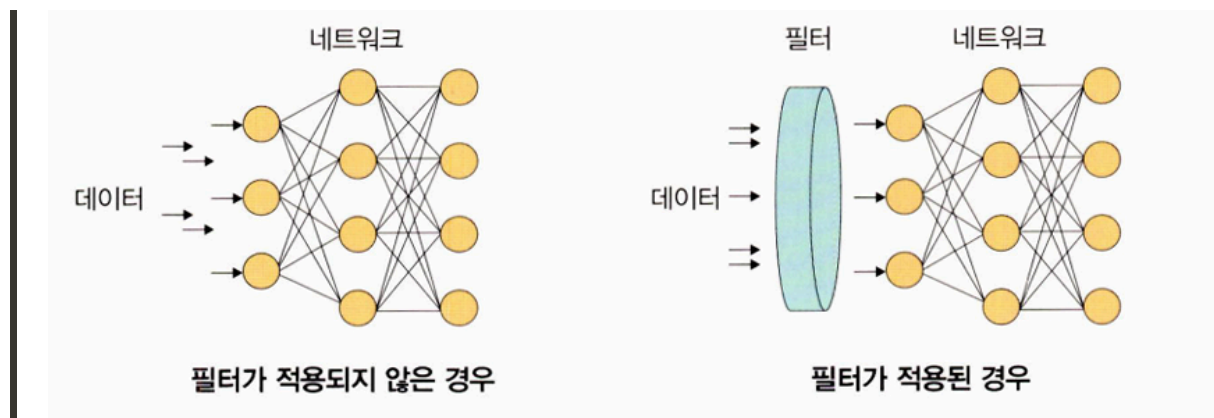


수식:

$$\frac{x - x_{min}}{x_{max} - x_{min}}$$

규제화(Regularization)

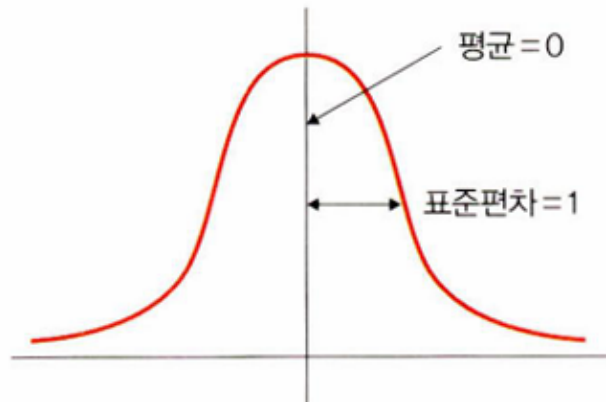
모델 복잡도를 줄이기 위해 제약을 두는 방법. 필터를 적용해 일부 데이터만 네트워크에 투입함으로써 빠르고 정확한 결과를 얻는 방식.



규제화 기법: 드롭아웃, 조기 종료 (각각 8.3.2, 8.3.3절에서 다룸)

표준화(Standardization)

데이터를 평균 0, 표준편차 1로 만드는 방법. z-스코어 정규화라고도 함.



수식:

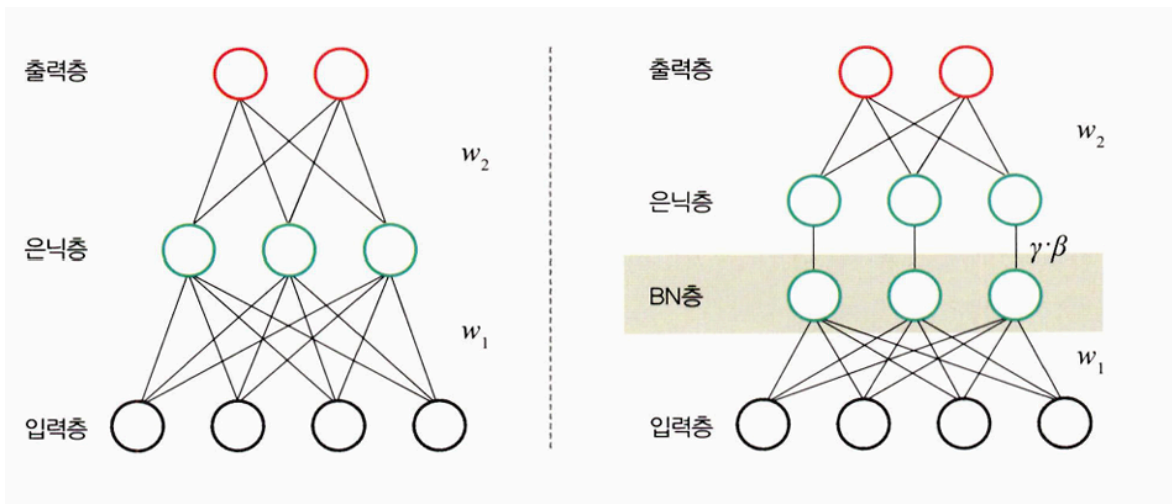
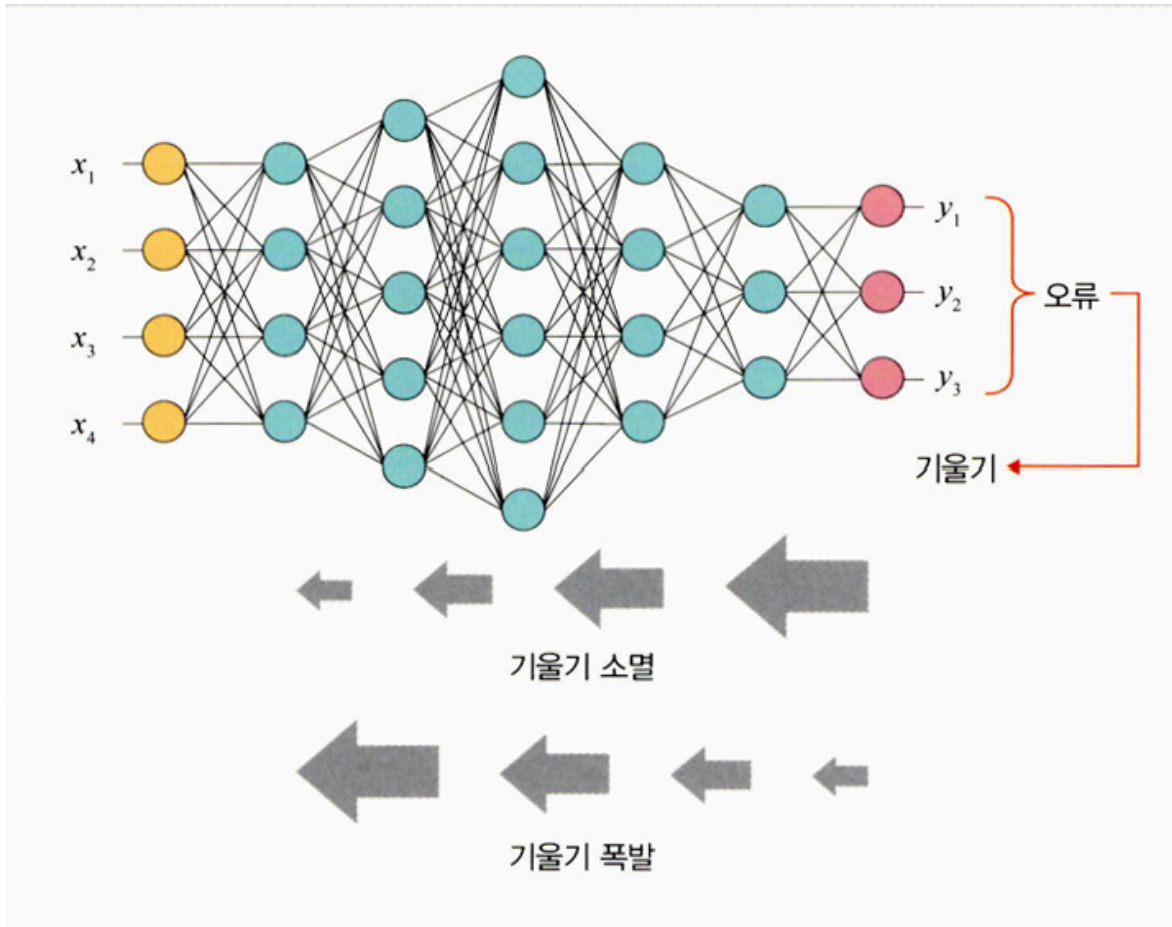
$$\frac{x-m}{\sigma}$$

배치 정규화(Batch Normalization)

2015년 논문 *"Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift"*에서 제안된 기법으로, 데이터 분포를 안정시켜 학습 속도를 높입니다.

해결하려는 문제: 기울기 소멸(gradient vanishing), 기울기 폭발(gradient exploding)

- **기울기 소멸:** 역전파 과정에서 기울기가 급격히 0에 가까워져 학습이 안 되는 현상
- **기울기 폭발:** 학습 과정에서 기울기가 급격히 커지는 현상



배치 정규화 수식 (미니 배치 단위)

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{----} \textcircled{1}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{----} \textcircled{2}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{----} \textcircled{3}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{----} \textcircled{4}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

① 미니 배치 평균 계산 → ② 분산/표준편차 계산 → ③ 정규화 수행 → ④ 스케일 조정(γ , β 는 학습되는 하이퍼파라미터)

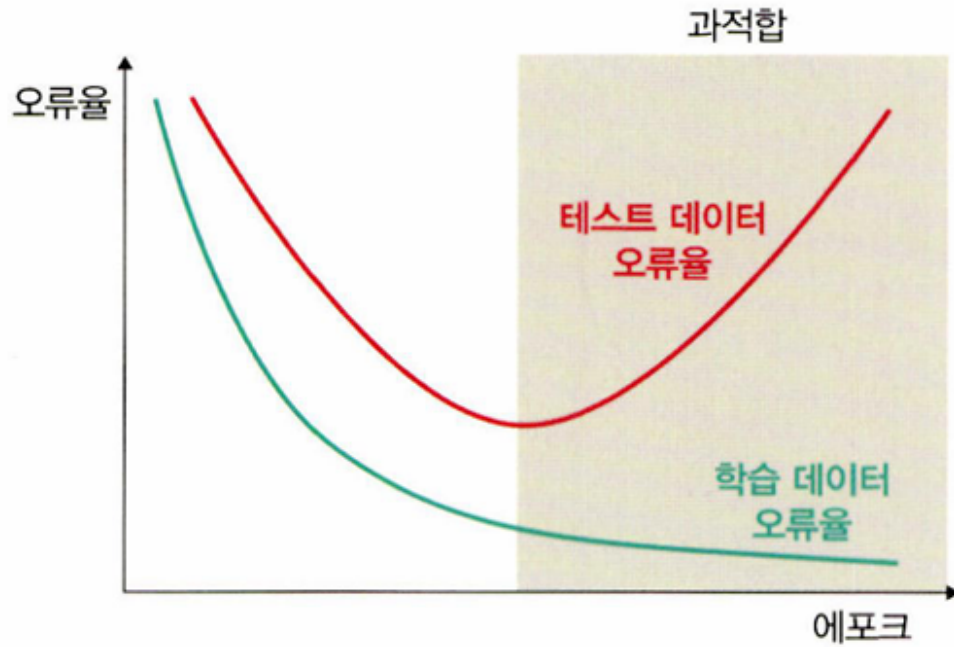
배치 정규화의 단점

1. 배치 크기가 작을 때 정규화 값이 기존과 다른 방향으로 훈련될 수 있음 (분산이 0이면 정규화 자체가 안 됨)
2. RNN은 계층별 미니 정규화가 필요해 모델이 더 복잡해질 수 있음

하지만 적용 시 성능이 좋아지는 경우가 많아 널리 사용됩니다.

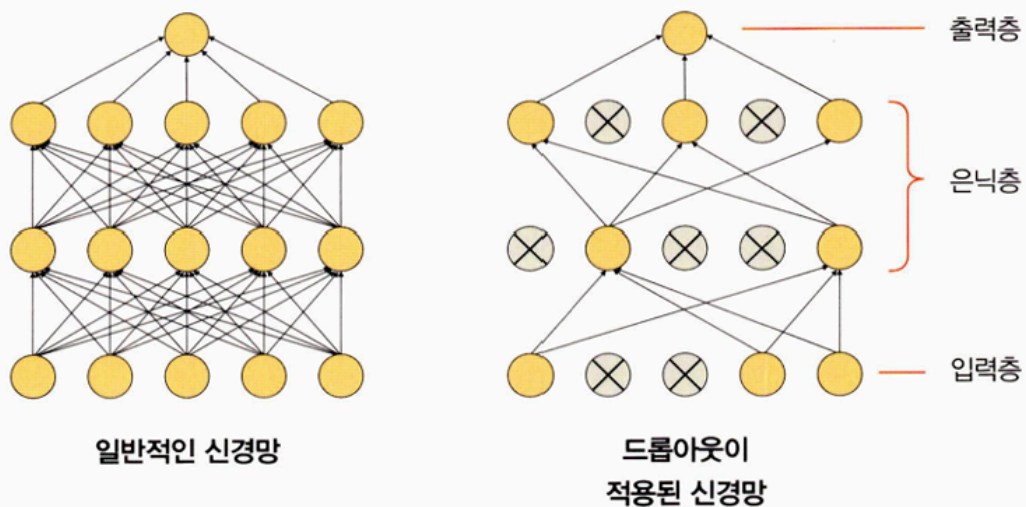
8.3.2 드롭아웃을 이용한 성능 최적화

- *과적합(overfitting)**이란 훈련 데이터셋을 과하게 학습하는 현상으로, 훈련 오류는 줄어들지만 테스트 오류는 어느 순간부터 증가합니다.



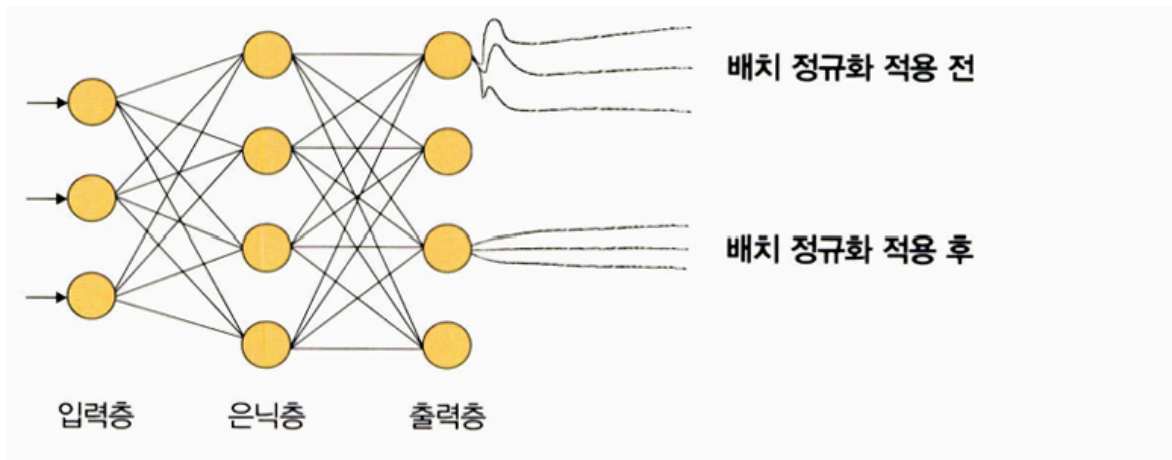
드롭아웃(Dropout): 훈련 시 일정 비율의 뉴런만 사용하고 나머지는 업데이트하지 않는 방법. 매 단계마다 사용하지 않는 뉴런을 바꿔가며 훈련합니다. 꺼진 노드는 신호를 전달하지 않으므로 과적합을 방지하는 효과가 있습니다.

▼ 그림 8-37 드롭아웃

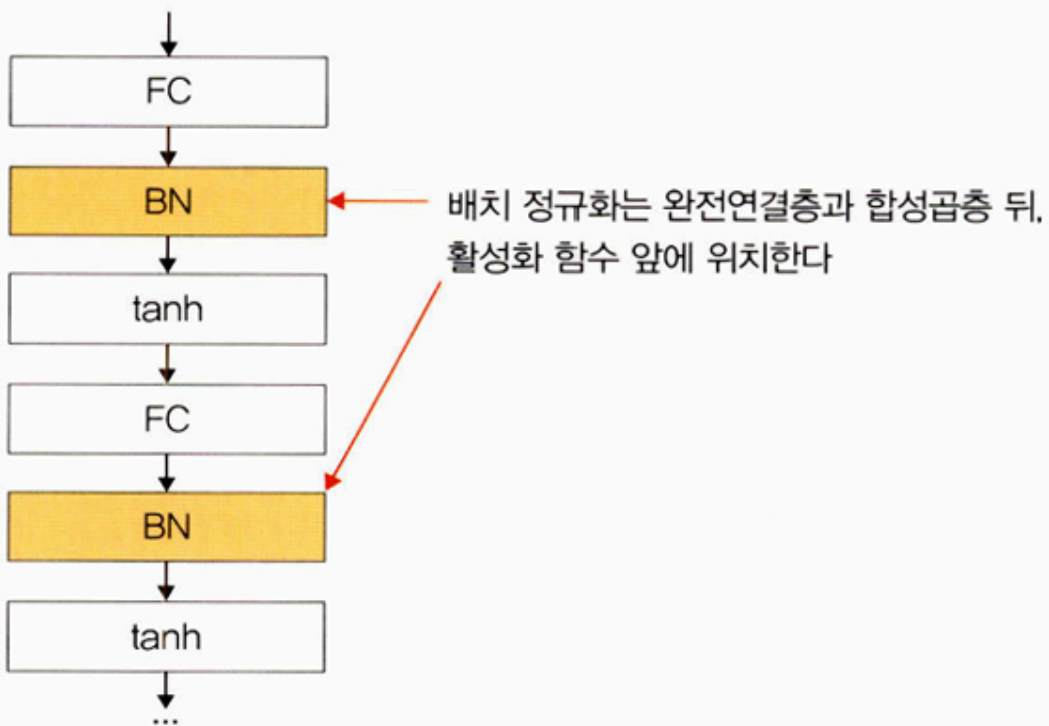


테스트 시에는 모든 노드를 사용하되, 노드 삭제 비율(드롭아웃 비율)을 곱해서 평가합니다. 훈련 시간이 길어지는 단점이 있지만 성능 향상에 자주 쓰입니다.

배치 정규화 위치: 완전연결층(FC)과 활성화 함수(tanh 등) 사이에 위치

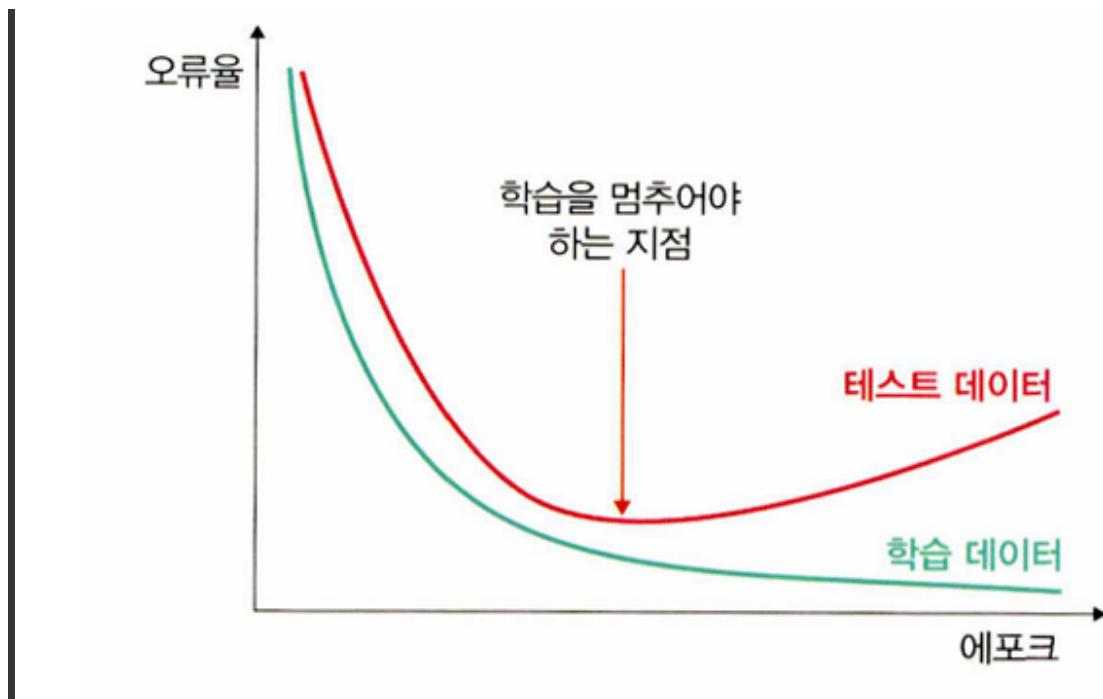


▼ 그림 8-40 배치 정규화 위치



8.3.3 조기 종료를 이용한 성능 최적화

조기 종료(Early Stopping): 과적합을 회피하는 규제 기법. 매 에포크마다 검증 데이터에 대한 오차(validation loss)를 측정하여 모델의 종료 시점을 제어합니다.



조기 종료는 학습을 언제 종료시킬지 결정할 뿐, **최고 성능을 보장하지는 않습니다.**

파라미터 설명

파라미터	설명
<code>mode</code>	'min'이면 오차 기준, 감소가 멈추면 학습률 조정 ('max'는 정확도처럼 클수록 좋은 지표용)
<code>patience</code>	학습률 업데이트 전 기다릴 에포크 수 (예: 5)
<code>factor</code>	학습률 감소 비율 (새 학습률 = 기존 학습률 × factor)
<code>min_lr</code>	학습률 하한선
<code>verbose</code>	1이면 학습률 조정 시 로그 출력

콜백 함수(Callback): 개발자가 명시적으로 호출하는 게 아니라 특정 이벤트 발생 시 자동으로 호출/처리되는 함수

케라스(`keras.callbacks`)는 `ModelCheckpoint`, `EarlyStopping` 을 기본 제공하지만, PyTorch는 조기 종료 함수를 직접 구현해야 합니다.

정리

기법	목적	핵심 효과
배치 정규화	기울기 소멸/폭발 방지	학습 속도 향상, 안정적 수렴
드롭아웃	과적합 방지	일반화 성능 향상

기법	목적	핵심 효과
학습률 스케줄러	학습 후반 미세 조정	정확도 향상, 안정적 학습
조기 종료	불필요한 학습 방지	자원 효율화 (성능 보장 X)

학습률 스케줄러와 조기 종료는 모델 성능 향상에 도움이 되지만, **모든 모델에 일괄 적용하기보다는 출력된 그래프를 해석해서 적용 여부를 결정하는 것이 중요합니다.**